



Cifrados de flujo - Desafíos en Seguridad

PROYECTO 1

Marco Ramirez

21032



Tabla de contenidos

<i>Tabla de contenidos</i>	<i>1</i>
<i>Luffy – XOR</i>	<i>3</i>
Introducción	3
Objetivo del reto	3
Reto.....	4
Flag walkthrough	4
Análisis del cifrado	7
Párrafo del poneglyphs	7
<i>Zoro – Rompiendo RC4</i>	<i>11</i>
Introducción	11
Objetivo del reto	11
Reto.....	12
Flag walkthrough	12
Análisis del cifrado	15
Párrafo del poneglyphs	18
<i>Usopp – XOR Cipher Custom</i>	<i>19</i>
Introducción	19
Objetivo del reto	19
Reto.....	20
Flag walkthrough	20

Análisis del cifrado	23
Párrafo de poneglyphs	24
<i>Nami – ChaCha20</i>	26
Introducción	26
Objetivo del reto	26
Reto	27
Flag walkthrough	27
Análisis del cifrado	30
Párrafo de poneglyphs	31
<i>Bibliografía</i>	32

Luffy – XOR

Introducción

En este reto se nos presenta un archivo cifrado utilizando la operación XOR con una clave basada en el carné del estudiante. El archivo contiene una imagen tipo *poneglyph* con un mensaje cifrado que debe ser descifrado para recuperar la flag y un párrafo oculto.

La operación XOR es fundamental en criptografía de flujo por su simplicidad, pero también por sus debilidades cuando la clave es predecible o reutilizada. El objetivo fue aprovechar esas vulnerabilidades para recuperar el mensaje original.

¿Qué es XOR?

La operación XOR (eXclusive OR) compara dos bits y devuelve 1 solo si son diferentes. En criptografía, se aplica **byte a byte** entre el texto original y una clave:

texto_cifrado = texto_original XOR clave

Objetivo del reto

- Descifrar un mensaje cifrado con XOR y una clave derivada del carné del estudiante.
1. Encontrar la imagen *poneglyph.jpeg* dentro del contenedor de Luffy.
 2. Extraer el texto cifrado (por ejemplo, desde *flag.txt*).

Reto

Flag walkthrough

1. Iniciamos sesión con luffy como con la contraseña de “onepiece”

su luffy

password: onepiece

```
nobody@650678f68447:/home/luffy$ su luffy
Password:
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

luffy@650678f68447:~$
```

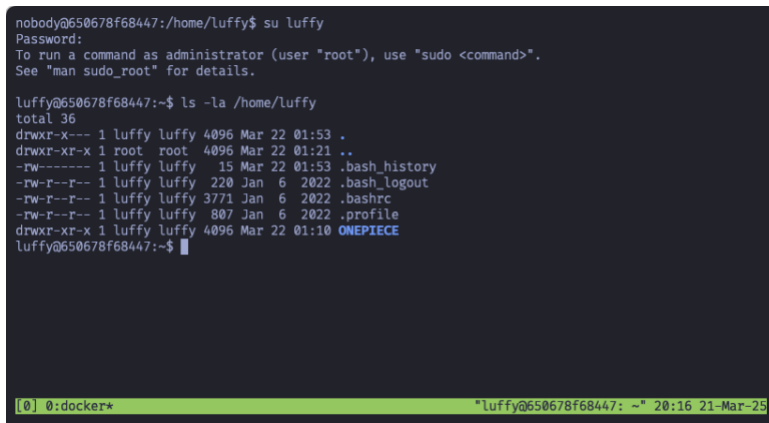
A terminal window with a dark background. The prompt is 'nobody@650678f68447:/home/luffy\$'. The user enters 'su luffy'. The prompt changes to 'luffy@650678f68447:~\$'. At the bottom, a green status bar shows '[0] 0:docker*' and '*luffy@650678f68447: ~* 20:14 21-Mar-25'.

2. Enumeración del directorio /home/luffy

- a. Ya dentro como el usuario luffy, listamos todos los archivos (incluso ocultos) de su directorio personal:
 - i. ls -la /home/luffy

```
nobody@650678f68447:/home/luffy$ su luffy
Password:
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

luffy@650678f68447:~$ ls -la /home/luffy
total 36
drwxr-x--- 1 luffy luffy 4096 Mar 22 01:53 .
drwxr-xr-x 1 root  root  4096 Mar 22 01:21 ..
-rw----- 1 luffy luffy  15 Mar 22 01:53 .bash_history
-rw-r--r-- 1 luffy luffy  220 Jan  6 2022 .bash_logout
-rw-r--r-- 1 luffy luffy 3771 Jan  6 2022 .bashrc
-rw-r--r-- 1 luffy luffy  807 Jan  6 2022 .profile
drwxr-xr-x 1 luffy luffy 4096 Mar 22 01:10 ONEPIECE
luffy@650678f68447:~$
```

A terminal window with a dark background. The prompt is 'nobody@650678f68447:/home/luffy\$'. The user enters 'su luffy'. The prompt changes to 'luffy@650678f68447:~\$'. The user enters 'ls -la /home/luffy'. The output shows the directory listing for /home/luffy, including files like .bash_history, .bash_logout, .bashrc, .profile, and a directory named ONEPIECE. At the bottom, a green status bar shows '[0] 0:docker*' and '*luffy@650678f68447: ~* 20:16 21-Mar-25'.

3. Búsqueda de archivos flag.txt

- a. Para localizar posibles flags dentro del sistema, utilizamos el comando find buscando archivos con el nombre flag.txt:
 - i. `find / -name flag.txt 2>/dev/null`

```
luffy@650678f68447:~/ONEPIECE/Whole_Cake_Island/Whole_Cake_Chateau/Casa_de_Katakuri$ find / -name flag.txt 2>/dev/null
/home/luffy/ONEPIECE/Fishman_Island/Gyoverly_Hills/Casa_de_Neptune/flag.txt
/home/luffy/ONEPIECE/Fishman_Island/Gyoncorde_Plaza/Casa_de_Neptune/flag.txt
/home/luffy/ONEPIECE/Alabasta/Rainbase/Casa_de_Crocodile/flag.txt
/home/luffy/ONEPIECE/Alabasta/Spiders_Cafe/Casa_de_Miss_All_Sunday/flag.txt
/home/luffy/ONEPIECE/Alabasta/Katorea/Casa_de_Kohza/flag.txt
/home/luffy/ONEPIECE/Wano/Flower_Capital/Casa_de_Kozuki_Momonosuke/flag.txt
/home/luffy/ONEPIECE/Pirate_Island/Pirates_Den/Casa_de_Shanks/flag.txt
/home/luffy/ONEPIECE/Pirate_Island/Pirates_Ship/Casa_de_Whitebeard/flag.txt
/home/luffy/ONEPIECE/Whole_Cake_Island/Liqueur_Island/Casa_de_Katakuri/flag.txt
/home/luffy/ONEPIECE/Whole_Cake_Island/Liqueur_Island/Casa_de_Big_Mom/flag.txt
/home/luffy/ONEPIECE/Whole_Cake_Island/Whole_Cake_Chateau/Casa_de_Pudding/flag.txt
luffy@650678f68447:~/ONEPIECE/Whole_Cake_Island/Whole_Cake_Chateau/Casa_de_Katakuri$
```

4. Script para probar el contenido de las flags

- a. Creamos un script en Bash para automatizar la búsqueda de la contraseña correcta en los archivos flag.txt y probarla contra un archivo .zip protegido llamado poneglyph.zip.

```
#!/bin/bash

# Ruta al zip protegido
ZIP_FILE="poneglyph.zip"

# Buscar todos los archivos flag.txt
find /home/luffy/ONEPIECE -name flag.txt | while read FLAG_PATH; do
    PASSWORD=$(cat "$FLAG_PATH")
    echo "Probando contraseña: $PASSWORD"

    # Probar descompresión silenciosa con contraseña
    unzip -P "$PASSWORD" -t "$ZIP_FILE" > /dev/null 2>&1

    if [ $? -eq 0 ]; then
        echo "✅ ¡Contraseña correcta encontrada!: $PASSWORD"
        # Extraer el zip
        unzip -P "$PASSWORD" "$ZIP_FILE"
        break
    fi
done
```

Nos auxiliamos de ChatGPT para crear el Script

- b. Identificación de texto cifrado y preparación para XOR
 - i. Al ejecutar el script, algunas líneas muestran mensajes normales, pero una de ellas contiene un texto en hexadecimal no legible

```
luffy@650678f68447:~/ONEPIECE/Whole_Cake_Island/Whole_Cake_Chateau/Casa_de_Katakuri$ sh script.sh
Probando contraseña: Has encontrado un lugar misterioso
Probando contraseña: 747d71746d500205500656570251500a035505040b57040201530356060505010757075407
Probando contraseña: Has encontrado un lugar abandonado
Probando contraseña: Has encontrado un lugar abandonado
Probando contraseña: Has encontrado un lugar peligroso
Probando contraseña: Has encontrado un lugar abandonado
Probando contraseña: Has encontrado un tesoro que apunta a Gyoncorde_Plaza
Probando contraseña: Has encontrado un enemigo y ha tenido que huir
Probando contraseña: Has encontrado un tesoro que apunta a Gyoncorde_Plaza
Probando contraseña: Has encontrado un tesoro que apunta a Gyoncorde_Plaza
Probando contraseña: Has encontrado un enemigo y ha tenido que luchar
luffy@650678f68447:~/ONEPIECE/Whole_Cake_Island/Whole_Cake_Chateau/Casa_de_Katakuri$
```

- c. Descifrado del texto con XOR en una línea de Python
 - i. Para descifrar el texto en hexadecimal, utilizamos una **one-liner en Python** que aplica XOR con una clave conocida (21032):

1. `python3 -c`

`'data=bytes.fromhex({Encrypted_Text}key="21032".encode());`

`print(''.join(chr(b ^ key[i % len(key)]) for i, b in enumerate(data)))'`

```
luffy@650678f68447:~/ONEPIECE/Whole_Cake_Island/Whole_Cake_Chateau/Casa_de_Katakuri$ python3 -c 'data=bytes.fromhex("747d71746d500205500656570251500a035505040b57040201530356060505010757075407"); key="21032".encode(); print(''.join(chr(b ^ key[i % len(key)]) for i, b in enumerate(data)))'
FLAG_b35c4df2bb82e669f413a2f57707d5f6
luffy@650678f68447:~/ONEPIECE/Whole_Cake_Island/Whole_Cake_Chateau/Casa_de_Katakuri$
```

Análisis del cifrado

El mensaje fue cifrado utilizando una operación XOR simple, empleando como clave el número de carné del estudiante (21032), codificado como cadena y repetido tantas veces como fuera necesario para coincidir con el largo del texto cifrado.

El commando utilizado realiza lo siguiente:

1. Convierte el texto cifrado desde hexadecimal a bytes.
2. Codifica la clave "21032" como bytes.
3. Aplica XOR byte a byte entre el mensaje y la clave repetida.
4. Imprime el mensaje descifrado (en este caso, la flag).

Párrafo del poneglyphs

5. Análisis de rutas sospechosas o archivos voluminosos
 - a. Como parte del reto, también buscamos archivos grandes o rutas sospechosas. Para ello usamos el comando:
 - i. `du -ah /home/luffy | sort -hr | head`

```
1120 1102 0:45 /mvrcentes/Library/CloudStorage/OneDrive-UVG/Documentos/Semestre_9/Cryptography/chall
enges_volumes/luffy_ctf /luffy_ctf rw,nosuid,nodev,relatime - fakeowner /run/host_mark/Users rw,fake
owner
/run/host_mark/Users /luffy_ctf fakeowner rw,nosuid,nodev,relatime,fakeowner 0 0
1120 1102 0:45 /mvrcentes/Library/CloudStorage/OneDrive-UVG/Documentos/Semestre_9/Cryptography/chall
enges_volumes/luffy_ctf /luffy_ctf rw,nosuid,nodev,relatime - fakeowner /run/host_mark/Users rw,fake
owner
/run/host_mark/Users /luffy_ctf fakeowner rw,nosuid,nodev,relatime,fakeowner 0 0
1120 1102 0:45 /mvrcentes/Library/CloudStorage/OneDrive-UVG/Documentos/Semestre_9/Cryptography/chall
enges_volumes/luffy_ctf /luffy_ctf rw,nosuid,nodev,relatime - fakeowner /run/host_mark/Users rw,fake
owner
^C
luffy@650678f68447:~$ du -ah /home/luffy | sort -hr | head
748K /home/luffy
728K /home/luffy/ONEPIECE
204K /home/luffy/ONEPIECE/Whole_Cake_Island
152K /home/luffy/ONEPIECE/Alabasta
124K /home/luffy/ONEPIECE/Pirate_Island
124K /home/luffy/ONEPIECE/Fishman_Island
116K /home/luffy/ONEPIECE/Wano
100K /home/luffy/ONEPIECE/Whole_Cake_Island/Whole_Cake_Chateau
76K /home/luffy/ONEPIECE/Whole_Cake_Island/Whole_Cake_Chateau/Casa_de_Katakuri
68K /home/luffy/ONEPIECE/Whole_Cake_Island/Whole_Cake_Chateau/Casa_de_Katakuri/poneglyph.zip
luffy@650678f68447:~$
```

[0] 0:docker* *luffy@650678f68447: ~* 20:17 21-Mar-23

- b. Extracción del archivo poneglyph.zip
 - i. Utilizamos el comando unzip para extraer el contenido del archivo comprimido poneglyph.zip. Se utilizó la contraseña **onepiece**, ya descubierta en pasos anteriores.
 - 1. unzip poneglyph.zip

```
luffy@650678f68447:~/ONEPIECE/Whole_Cake_Island/Whole_Cake_Chateau/Casa_de_Katakuri$ unzip poneglyph
.zip
Archive:  poneglyph.zip
[poneglyph.zip] challenges/luffy/poneglyph.jpeg password:
replace challenges/luffy/poneglyph.jpeg? [y]es, [n]o, [A]ll, [N]one, [r]ename: N
luffy@650678f68447:~/ONEPIECE/Whole_Cake_Island/Whole_Cake_Chateau/Casa_de_Katakuri$
```

- ii. Ubicación del archivo extraído
 - 1. Después de descomprimir poneglyph.zip, se creó una carpeta llamada challenges, y dentro de ella, una subcarpeta luffy que contiene la imagen extraída

```
luffy@650678f68447:~/ONEPIECE/Whole_Cake_Island/Whole_Cake_Chateau/Casa_de_Katakuri$ ls
challenges  poneglyph.zip  script.sh
luffy@650678f68447:~/ONEPIECE/Whole_Cake_Island/Whole_Cake_Chateau/Casa_de_Katakuri$ ls challenges/luffy/poneglyph.jpeg
challenges/luffy/poneglyph.jpeg
luffy@650678f68447:~/ONEPIECE/Whole_Cake_Island/Whole_Cake_Chateau/Casa_de_Katakuri$
```

6. Copiar la imagen del contenedor a nuestra computadora

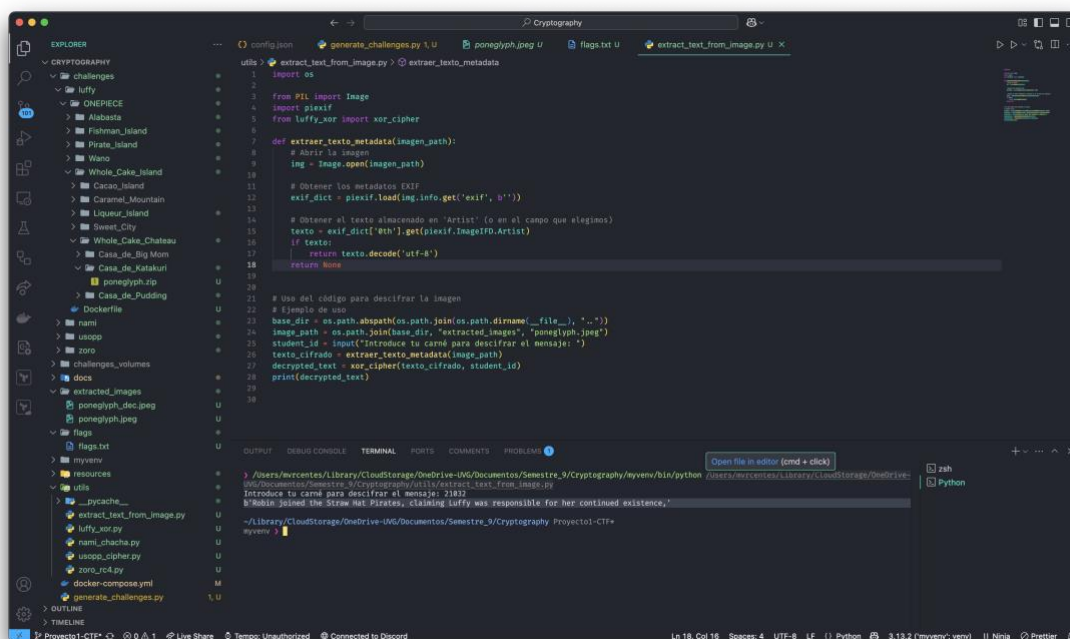
- a. Una vez localizada y extraída la imagen poneglyph.jpeg, la copiamos desde el contenedor Docker hacia un directorio local usando el comando `docker cp`.

`luffy_challenge:/home/luffy/ONEPIECE/Whole_Cake_Island/Whole_Cake_Chateau/Casa_de_Katakuri/challenges/luffy/poneglyph.jpeg .`

```
~/Library/CloudStorage/OneDrive-UVG/Documentos/Semestre_9/Cryptography/extracted_images Proyecto1-CTF*  
> docker cp luffy_challenge:/home/luffy/ONEPIECE/Whole_Cake_Island/Whole_Cake_Chateau/Casa_de_Katakuri/challenges/luffy/poneglyph.jpeg .  
> docker cp luffy_challenge:/home/luffy/ONEPIECE/Whole_Cake_Island/Whole_Cake_Chateau/Casa_de_Katakuri/challenges/luffy/poneglyph.jpeg .  
Successfully copied 1.54kB to /Users/mvrcntes/Library/CloudStorage/OneDrive-UVG/Documentos/Semestre_9/  
  
~/Library/CloudStorage/OneDrive-UVG/Documentos/Semestre_9/Cryptography/extracted_images Proyecto1-CTF*  
> █  
[1] 0:zsh* "Marcos-MacBook-Pro-6." 12:02 25-Mar-25
```

7. Ejecución del script para extraer el texto de la imagen

- Ejecutamos el script proporcionado por el proyecto para extraer el texto de la imagen usando el siguiente código:



```

1 import os
2
3 from PIL import Image
4 import piexif
5 from luffy_xor import xor_cipher
6
7 def extraer_texto_metadata(imagen_path):
8     # Abrir la imagen
9     img = Image.open(imagen_path)
10
11     # Obtener los metadatos EXIF
12     exif_dict = piexif.load(img.info.get('exif', b''))
13
14     # Obtener el texto almacenado en 'Artist' (o en el campo que elegimos)
15     texto = exif_dict['Eth'].get(piexif.ImageIFD.Artist)
16     if texto:
17         return texto.decode('utf-8')
18     return None
19
20 # Uso del código para descifrar la imagen
21 # Ejemplo de uso
22 base_dir = os.path.abspath(os.path.join(os.path.dirname(__file__), '..'))
23 image_path = os.path.join(base_dir, 'extracted_images', 'poneglyph.jpg')
24 student_id = input("Introduce tu carné para descifrar el mensaje: ")
25 texto_cifrado = extraer_texto_metadata(image_path)
26 decrypted_text = xor_cipher(texto_cifrado, student_id)
27 print(decrypted_text)
28

```

OUTPUT DEBUG CONSOLE TERMINAL PORTS COMMENTS PROBLEMS

~/Library/CloudStorage/OneDrive-UMG/Documentos/Semestre_9/Cryptography/Project01-CTF+ myenv

Introduce tu carné para descifrar el mensaje: 2182

b'Robin joined the Straw Hat Pirates, claiming Luffy was responsible for her continued existence,'

Texto encontrado: b'Robin joined the Straw Hat Pirates, claiming Luffy was responsible for her continued existence,'

Zoro – Rompiendo RC4

Introducción

En este reto se nos presenta un mensaje cifrado utilizando el algoritmo de flujo RC4, el cual utiliza una clave secreta (en este caso, derivada del carné del estudiante) para generar un flujo de claves pseudoaleatorio. Este flujo se combina con el texto original mediante la operación XOR.

Aunque RC4 fue ampliamente utilizado en el pasado, actualmente se considera inseguro debido a diversas debilidades conocidas. El objetivo del reto es identificar esas vulnerabilidades y recuperar el mensaje original.

¿Qué es RC4?

RC4 (Rivest Cipher 4) es un algoritmo de cifrado de flujo que utiliza una clave para inicializar un estado interno de 256 bytes. A partir de ese estado, se genera un flujo de claves pseudoaleatorio (keystream), que se aplica al texto original usando XOR:

`texto_cifrado = texto_original XOR flujo_de_claves`

Objetivo del reto

- Descifrar un mensaje cifrado con RC4 utilizando una clave derivada del carné del estudiante.
1. Encontrar el archivo flag.txt dentro del contenedor de Zoro.
 2. Aplicar el algoritmo RC4 con la clave "21032" para generar el flujo de claves.
 3. Realizar la operación XOR entre el flujo de claves generado y el texto cifrado.
 4. Recuperar el mensaje original y extraer la flag.

Reto

Flag walkthrough

1. Iniciar sesión como zoro

- a. Para este reto, accedemos con el usuario zoro utilizando como contraseña el **flag** obtenido del reto anterior:

- i. su zoro
- ii. # Password: FLAG_b35c4df2bb82e669f413a2f57707d5f6

2. Exploración del directorio de zoro

- a. Una vez dentro del entorno del usuario zoro, listamos todos los archivos, incluyendo los ocultos, en su directorio personal:

- i. ls -la

```
zoro@4652857476c9:~$ ls -la
total 36
drwxr-x--- 1 zoro zoro 4096 Mar 22 01:21 .
drwxr-xr-x 1 root root 4096 Mar 22 01:21 ..
-rw-r--r-- 1 zoro zoro 220 Jan 6 2022 .bash_logout
-rw-r--r-- 1 zoro zoro 3771 Jan 6 2022 .bashrc
-rw-r--r-- 1 zoro zoro 807 Jan 6 2022 .profile
drwxr-xr-x 1 zoro zoro 4096 Mar 22 01:10 ONEPIECE
zoro@4652857476c9:~$
```

[0] 0:docker* "zoro@4652857476c9: ~" 19:27 25-Mar-25

Encontramos un directorio el cual vamos a estar explorando más adelante

3. Búsqueda de archivos relevantes dentro de ONEPIECE/

- a. Realizamos una búsqueda recursiva dentro del directorio ONEPIECE/ para localizar archivos que puedan ser importantes para el reto: textos, archivos comprimidos o cifrados.

- i. `find ONEPIECE/ -type f \( -iname "*.txt" -o -iname "*.zip" -o -iname "*.enc" \) 2>/dev/null,`

```
zoro@4652857476c9:~$ find ONEPIECE/ -type f \( -iname "*.txt" -o -iname "*.zip" -o -iname "*.enc" \) 2>/dev/null
ONEPIECE/Fishman_Island/Mermaid_Cove/Casa_de_Otohome/flag.txt
ONEPIECE/Fishman_Island/Coral_Hill/Casa_de_Shirahoshi/flag.txt
ONEPIECE/Wano/Kuri/Casa_de_Kozuki_Hiyori/flag.txt
ONEPIECE/Wano/Onigashima/Casa_de_Kaido/flag.txt
ONEPIECE/Wano/Ringo/Casa_de_Kozuki_Hiyori/flag.txt
ONEPIECE/Wano/Udon/Casa_de_Oden/flag.txt
ONEPIECE/Wano/Udon/Casa_de_Kozuki_Hiyori/flag.txt
ONEPIECE/Pirate_Island/Pirates_Cove/Casa_de_Gold_Roger/flag.txt
ONEPIECE/Pirate_Island/Pirates_Hideout/Casa_de_Shanks/flag.txt
ONEPIECE/Pirate_Island/Pirates_Hideout/Casa_de_Gold_Roger/flag.txt
ONEPIECE/Whole_Cake_Island/Caramel_Mountain/Casa_de_Pudding/flag.txt
ONEPIECE/Whole_Cake_Island/Liqueur_Island/Casa_de_Pudding/poneglyph.zip
zoro@4652857476c9:~$
```

[0] 0:docker* "zoro@4652857476c9: ~" 19:32 25-Mar-25

4. Creación de un script para revisar el contenido de archivos .txt

- a. Para validar rápidamente qué contiene cada archivo flag.txt encontrado, creamos un pequeño script en Python que recorre todo el directorio ONEPIECE/, abre los archivos .txt y muestra sus primeros 300 caracteres:

```
import os

base_path = "ONEPIECE"

for root, dirs, files in os.walk(base_path):
    for file in files:
        if file.endswith(".txt"):
            file_path = os.path.join(root, file)
            print(f"\n {file_path}")
            print("-" * 60)
            try:
                with open(file_path, "r", encoding="utf-8", errors="ignore") as f:
                    content = f.read()
                    print(content[:300]) # Mostrar primeros 300 caracteres
            except Exception as e:
                print(f"Error al leer {file_path}: {e}")
```

4.1. Ejecución del script de lectura de archivos .txt

- a. Ejecutamos el script script.py previamente creado, el cual recorre todos los archivos .txt dentro del directorio ONEPIECE y muestra los primeros 300 caracteres de su contenido.
- a. `python3 script.py`

```

zoro@4652857476c9:~$ ls
ONEPIECE script.py
zoro@4652857476c9:~$ python3 script.py
📄 ONEPIECE/Fishman_Island/Mermaid_Cove/Casa_de_Otohime/flag.txt
-----
Has encontrado un lugar peligroso
📄 ONEPIECE/Fishman_Island/Coral_Hill/Casa_de_Shirahoshi/flag.txt
-----
Has encontrado un tesoro que apunta a Pirates_Cove
📄 ONEPIECE/Wano/Kuri/Casa_de_Kozuki_Hiyori/flag.txt
-----
Has encontrado un tesoro que apunta a Pirates_Cove
📄 ONEPIECE/Wano/Onigashima/Casa_de_Kaido/flag.txt
-----
Has encontrado un obstáculo y hay que superarlo
📄 ONEPIECE/Wano/Ringo/Casa_de_Kozuki_Hiyori/flag.txt
-----
Has encontrado un amigo y han decidido viajar juntos
📄 ONEPIECE/Wano/Udon/Casa_de_Oden/flag.txt
-----
Has encontrado un lugar peligroso
📄 ONEPIECE/Wano/Udon/Casa_de_Kozuki_Hiyori/flag.txt
-----
Has encontrado un objeto misterioso y no sabes qué es
📄 ONEPIECE/Pirate_Island/Pirates_Cove/Casa_de_Gold_Roger/flag.txt
-----
068b673b6b5705abb8983c0f29a5f848e76ebc8067bce803a9a7df83cdbe201468450a0f7b
📄 ONEPIECE/Pirate_Island/Pirates_Hideout/Casa_de_Shanks/flag.txt
-----
Has encontrado una pista que apunta a Pirates_Cove
📄 ONEPIECE/Pirate_Island/Pirates_Hideout/Casa_de_Gold_Roger/flag.txt
-----
Has encontrado un lugar peligroso
📄 ONEPIECE/Whole_Cake_Island/Caramel_Mountain/Casa_de_Pudding/flag.txt
-----
Te cansaste de buscar y te has quedado dormido
zoro@4652857476c9:~$
[0] 0:docker* "zoro@4652857476c9: ~" 19:36 25-Mar-25

```

Notamos que un texto que no es legible por lo que probamos descriptarlo

5. Generar un script para descifrar el texto con RC4

- a. Para descifrar el texto cifrado encontrado en el paso anterior, se utilizó el algoritmo **RC4**. Creamos un script en Python usando pycryptodome que utiliza como clave nuestro número de carné.

```
from Crypto.Cipher import ARC4

# Reemplaza con tu número de carné
student_id = "21032"
key = student_id.encode()

# Hex cifrado desde el archivo
hex_cipher = "068b673b6b5705abb8983c0f29a5f848e76ebc8067bce803a9a7df83cdbc201468450a0f7b"

# Convertir de hex a bytes
cipher_bytes = bytes.fromhex(hex_cipher)

# Inicializar RC4 con tu clave
cipher = ARC4.new(key)

# Descifrar
plaintext = cipher.decrypt(cipher_bytes)

# Mostrar resultado
print("\n🚩 Texto descifrado:")
print(plaintext.decode(errors="ignore"))
```

Análisis del cifrado

El mensaje estaba cifrado utilizando el algoritmo de flujo RC4, que genera un flujo de claves pseudoaleatorio (keystream) a partir de una clave secreta. En este caso, la clave era el número de carné del estudiante: 21032.

Para descifrar el texto, se replicó el comportamiento de RC4 usando la misma clave y se aplicó XOR entre el flujo generado y el texto cifrado, lo cual reveló la flag.

El uso de una clave corta y predecible, como el número de carné sin ninguna transformación o salting, representa una debilidad importante en la implementación.

6. Ejecutar el script de descifrado

- a. Ejecutamos el script `decrypt_rc4.py` que creamos anteriormente para descifrar el texto cifrado con RC4:

- i. `python3 decrypt_rc4.py`

```
zoro@4652857476c9:~$ python3 decrypt_rc4.py

🐼 Texto descifrado:
FLAG_df6996554cf497631b4b13402611f876
zoro@4652857476c9:~$ █

[0] 0:docker* "zoro@4652857476c9: ~" 19:48 25-Mar-25
```

Flag encontrda: FLAG_df6996554cf497631b4b13402611f876

7. Descompresión del archivo poneglyph.zip encontrado

- a. En el **paso 3**, identificamos el archivo comprimido poneglyph.zip en la siguiente ruta:
 - i. ONEPIECE/Whole_Cake_Island/Liqueur_Island/Casa_de_Pudding/poneglyph.zip
- b. Procedemos a descomprimirlo usando el comando unzip:
 - i. unzip poneglyph.zip

```
zoro@4652857476c9:~/ONEPIECE/Whole_Cake_Island/Liqueur_Island/Casa_de_Pudding$ unzip poneglyph.zip
Archive:  poneglyph.zip
[poneglyph.zip] challenges/zoro/poneglyph.jpeg password:
replace challenges/zoro/poneglyph.jpeg? [y]es, [n]o, [A]ll, [N]one, [r]ename: y
  inflating: challenges/zoro/poneglyph.jpeg
zoro@4652857476c9:~/ONEPIECE/Whole_Cake_Island/Liqueur_Island/Casa_de_Pudding$
```

[0] 0:docker* "zoro@4652857476c9: ~/" 20:10 25-Mar-25

Párrafo del poneglyphs

8. Copiar la imagen extraída del contenedor a nuestra máquina local

- a. Después de descomprimir la imagen poneglyph.jpeg, la copiamos desde el contenedor Docker al equipo local usando el siguiente comando:

- i. docker cp

```
zoro_challenge:/home/zoro/ONEPIECE/Whole_Cake_Island/Liqueur_Island/Casa_de_Pudding/challenges/zoro/poneglyph.jpeg
./poneglyph_zoro.jpg
```

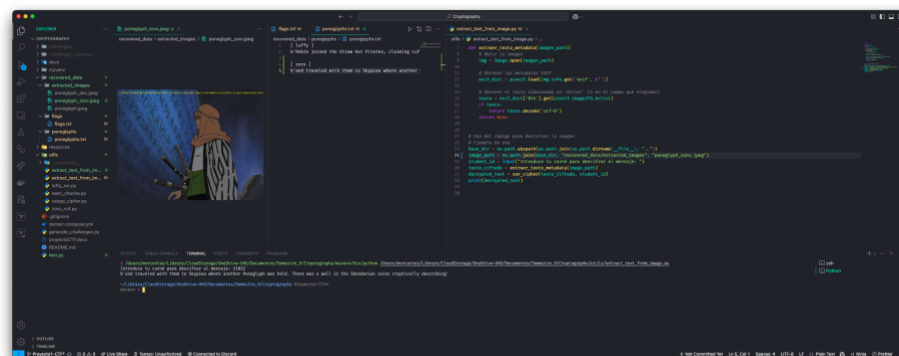
```
> docker cp zoro_challenge:/home/zoro/ONEPIECE/Whole_Cake_Island/Liqueur_Island/Casa_de_Pudding/challenges/zoro/poneglyph.jpeg ./poneglyph_zoro.jpg
Successfully copied 57.3kB to /Users/mvrcntes/Library/CloudStorage/OneDrive-UVG/Documentos/Semestre_9/Cryptography/recovered_data/extracted_images/poneglyph_zoro.jpg

~/Library/CloudStorage/OneDrive-UVG/Documentos/Semestre_9/Cryptography/recovered_data/extracted_images Proyecto1-CTF+
> █

[2] 0:zsh* *Marcos-MacBook-Pro-6.* 20:20 25-Mar-25
```

9. Ejecutar el script para extraer el texto de la imagen

- a. Usamos el script proporcionado por el proyecto para analizar la imagen poneglyph_zoro.jpg y extraer cualquier mensaje oculto, ya sea en los metadatos o incrustado visualmente.



Texto extraído: b'and traveled with them to Skypiea where another Poneglyph was held.

There was a wall in the Shandorian ruins cryptically describing'

Usopp – XOR Cipher Custom

Introducción

En este reto se nos presenta un mensaje cifrado utilizando una versión personalizada del cifrado de flujo XOR, en combinación con un generador de números pseudoaleatorios (PRNG) débil.

El flujo de claves fue generado dinámicamente a partir de una semilla (posiblemente derivada del carné del estudiante), y se aplicó al mensaje original mediante XOR.

Este tipo de implementación suele ser vulnerable si el PRNG es predecible o si no se implementa adecuadamente el manejo de claves.

El reto simula una situación en la que, aunque el algoritmo de cifrado no es estándar, sus debilidades pueden explotarse para recuperar el texto original. En este caso, se aplicó fuerza bruta sobre la semilla del generador hasta encontrar una que permitiera descifrar correctamente el mensaje.

Objetivo del reto

- Descifrar un mensaje cifrado con un flujo generado a partir de un PRNG personalizado.
 1. Identificar el texto cifrado dentro del contenedor de Usopp.
 2. Entender o suponer cómo se generó el flujo de clave (mediante análisis o lectura del código del generador).

Reto

Flag walkthrough

1. Iniciar sesión como el usuario usopp

- a. Iniciamos sesión con el usuario correspondiente al reto:

- i. su usopp

Contraseña utilizada:

La flag obtenida en el reto anterior:

FLAG_df699555cf497631b4b13402611f8f76

```
nobody@11bed3c57193:/home/usopp$ su usopp
Password:
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

usopp@11bed3c57193:~$
```

[3] 0:docker* "usopp@11bed3c57193: ~" 08:35 31-Mar-25

2. Exploración de los directorios

- a. Al igual que en los retos anteriores, exploramos el directorio de usopp para verificar su contenido:

- i. ls -la

```
usopp@11bed3c57193:~$ ls -la
total 36
drwxr-x--- 1 usopp usopp 4096 Mar 22 01:21 .
drwxr-xr-x 1 root  root  4096 Mar 22 01:21 ..
-rw-r--r-- 1 usopp usopp  220 Jan  6 2022 .bash_logout
-rw-r--r-- 1 usopp usopp 3771 Jan  6 2022 .bashrc
-rw-r--r-- 1 usopp usopp  807 Jan  6 2022 .profile
drwxr-xr-x 1 usopp usopp 4096 Mar 22 01:10 ONEPIECE
usopp@11bed3c57193:~$
```

[3] 0:docker* "usopp@11bed3c57193: ~" 08:36 31-Mar-25

3. Exploración de archivos dentro del directorio ONEPIECE

- a. Realizamos una búsqueda dentro del directorio ONEPIECE/ para localizar archivos que podrían ser relevantes para el reto, como archivos .txt, .zip o .enc:

i. `find ONEPIECE/ -type f \( -iname "*.txt" -o -iname "*.zip" -o -iname "*.enc" \) 2>/dev/null`

```
usopp@11bed3c57193:~$ find ONEPIECE/ -type f \( -iname "*.txt" -o -iname "*.zip" -o -iname "*.enc" \) 2>/dev/null
ONEPIECE/Fishman_Island/Coral_Mansion/Casa_de_Shirahoshi/flag.txt
ONEPIECE/Alabasta/Rainbase/Casa_de_Vivi/flag.txt
ONEPIECE/Alabasta/Spiders_Cafe/Casa_de_Nico_Robin/flag.txt
ONEPIECE/Alabasta/Nanohana/Casa_de_Pell/flag.txt
ONEPIECE/Wano/Onigashima/Casa_de_Kaido/flag.txt
ONEPIECE/Wano/Udon/Casa_de_Oden/flag.txt
ONEPIECE/Wano/Udon/Casa_de_Kozuki_Hiyori/flag.txt
ONEPIECE/Pirate_Island/Pirates_Ship/Casa_de_Shanks/flag.txt
ONEPIECE/Whole_Cake_Island/Caramel_Mountain/Casa_de_Katakuri/flag.txt
ONEPIECE/Whole_Cake_Island/Sweet_City/Casa_de_Big_Mom/flag.txt
ONEPIECE/Whole_Cake_Island/Whole_Cake_Chateau/Casa_de_Pudding/flag.txt
ONEPIECE/Whole_Cake_Island/Whole_Cake_Chateau/Casa_de_Big.zip
usopp@11bed3c57193:~$
```

4. Revisión del contenido de todos los archivos .txt

- a. Creamos y ejecutamos un script en Python que recorre recursivamente todos los archivos .txt dentro del directorio ONEPIECE y muestra su contenido, con el fin de detectar cadenas distintas o cifradas.

```
import os

base_path = "ONEPIECE"

for root, dirs, files in os.walk(base_path):
    for file in files:
        if file.endswith(".txt"):
            file_path = os.path.join(root, file)
            print(f"\n📄 {file_path}")
            print("-" * 60)
            try:
                with open(file_path, "r", encoding="utf-8", errors="ignore") as f:
                    content = f.read()
                    print(content[:300]) # Mostrar primeros 300 caracteres
            except Exception as e:
                print(f"Error al leer {file_path}: {e}")
```

```

usopp@11bed3c57193:~$ python3 script.py
ONEPIECE/Fishman_Island/Coral_Mansion/Casa_de_Shirahoshi/flag.txt
Has encontrado un obstáculo y hay que superarlo
ONEPIECE/Alabasta/Rainbase/Casa_de_Vivi/flag.txt
Has encontrado un objeto misterioso y no sabes qué es
ONEPIECE/Alabasta/Spiders_Cafe/Casa_de_Nico_Robin/flag.txt
Has encontrado una pista que apunta a Whole_Cake_Chateau
ONEPIECE/Alabasta/Nanohana/Casa_de_Pell/flag.txt
Has encontrado un amigo y han decidido viajar juntos
ONEPIECE/Wano/Onigashima/Casa_de_Kaido/flag.txt
Te cansaste de buscar y te has quedado dormido
ONEPIECE/Wano/Udon/Casa_de_Oden/flag.txt
Has encontrado un obstáculo y hay que superarlo
ONEPIECE/Wano/Udon/Casa_de_Kozuki_Hiyori/flag.txt
Has encontrado un enemigo y ha tenido que huir
ONEPIECE/Pirate_Island/Pirates_Ship/Casa_de_Shanks/flag.txt
Has encontrado un lugar abandonado
ONEPIECE/Whole_Cake_Island/Caramel_Mountain/Casa_de_Katakuri/flag.txt
Has encontrado una pista que apunta a Whole_Cake_Chateau
ONEPIECE/Whole_Cake_Island/Sweet_City/Casa_de_Big_Mom/flag.txt
Has encontrado un enemigo y ha tenido que luchar
ONEPIECE/Whole_Cake_Island/Whole_Cake_Chateau/Casa_de_Pudding/flag.txt
a77742694e4807d31f3b3b308196d52d196808649d4e198d421572e915afc68dea4652dd68
usopp@11bed3c57193:~$
[3] @:docker* usopp@11bed3c57193: ~$ 08:47 31-Mar-25

```

5. Usar la función personalizada para descifrar el texto

- Con la cadena hexadecimal identificada en el paso anterior, utilizamos una función `decrypt()` proporcionada por el proyecto dentro del módulo `usopp_cipher` para aplicar un cifrado XOR personalizado.

```

extract_text_from_image.py test.py U X
test.py
1 from utils.usopp_cipher import decrypt
2
3 # llama la función decrypt con tu flag
4 decrypt("a77742694e4807d31f3b3b308196d52d196808649d4e198d421572e915afc68dea4652dd68")

OUTPUT DEBUG CONSOLE TERMINAL PORTS COMMENTS PROBLEMS
~/Library/CloudStorage/OneDrive-UVG/Documentos/Semestre_9/Cryptography/myvenv/bin/python ~/Library/CloudStorage/OneDrive-UVG/Documentos/Semestre_9/Cryptography/test.py
Semilla encontrada: 1234
Texto descifrado: FLAG_b5ff3480a8a752cd18ca8ec5508b945a
~/Library/CloudStorage/OneDrive-UVG/Documentos/Semestre_9/Cryptography Proyecto1-CTF*
myvenv >

```

Flag encontrada: FLAG_b5ff3480a8a752cd18ca8ec5508b945a

Análisis del cifrado

Este reto utilizó un cifrado personalizado basado en XOR con un flujo de claves generado por un PRNG (Generador de Números Pseudoaleatorios). La implementación se encuentra en el archivo `usopp_cipher.py`, donde se utiliza la función `random.seed()` con una semilla secreta (número entero) para inicializar el generador de claves.

A diferencia de un simple XOR con clave fija, aquí la clave cambia en cada posición, simulando un cifrado de flujo real. Sin embargo, al usar un PRNG débil y sin salt ni mecanismos de protección, el flujo generado es totalmente determinista y predecible si se conoce (o adivina) la semilla usada.

Párrafo de poneglyphs

6. Descomprimir el archivo .zip con la flag como contraseña

- a. Procedemos a descomprimir el archivo Casa_de_Big.zip encontrado previamente.

Para ello usamos como contraseña la **flag obtenida del reto anterior**:

- i. unzip Casa_de_Big.zip
- ii. **Contraseña usada:** FLAG_91df3f5ffbf7a25c1c041ecb54689a50

```
usopp@11bed3c57193:~/ONEPIECE/Whole_Cake_Island/Whole_Cake_Chateau$ unzip Casa_de_Big.zip
Archive: Casa_de_Big.zip
[Casa_de_Big.zip] challenges/usopp/poneglyph.jpeg password:
password incorrect--reenter:
  inflating: challenges/usopp/poneglyph.jpeg
usopp@11bed3c57193:~/ONEPIECE/Whole_Cake_Island/Whole_Cake_Chateau$
usopp@11bed3c57193:~/ONEPIECE/Whole_Cake_Island/Whole_Cake_Chateau$ ls
'Casa_de_Big Mom' Casa_de_Big.zip Casa_de_Katakuri Casa_de_Pudding challenges
usopp@11bed3c57193:~/ONEPIECE/Whole_Cake_Island/Whole_Cake_Chateau$
```

[3] 0:docker* "usopp@11bed3c57193: ~" 10:19 31-Mar-25

7. Copiar la imagen descomprimida al equipo local

- a. Una vez descomprimida la imagen poneglyph.jpeg desde el archivo

Casa_de_Big.zip, procedemos a copiarla desde el contenedor Docker al sistema local con el comando docker cp.

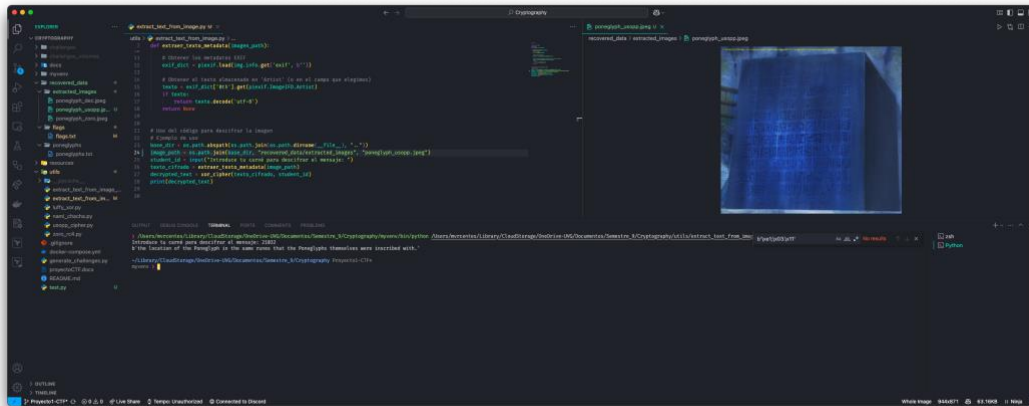
- i. docker cp
- usopp_challenge:/home/usopp/ONEPIECE/Whole_Cake_Island/Whole_Cake_Chateau/challenges/usopp/poneglyph.jpeg poneglyph_usopp.jpeg

```
~/Library/CloudStorage/OneDrive-UVG/Documentos/Semestre_9/Cryptography/recovered_data/extracted_images Proyecto1-CTF*
> docker cp usopp_challenge:/home/usopp/ONEPIECE/Whole_Cake_Island/Whole_Cake_Chateau/challenges/usopp ./poneglyph_usopp.jpeg
> docker cp usopp_challenge:/home/usopp/ONEPIECE/Whole_Cake_Island/Whole_Cake_Chateau/challenges/usopp ./poneglyph_usopp.jpeg
Successfully copied 67.1kB to /Users/mvrcentes/Library/CloudStorage/OneDrive-UVG/Documentos/Semestre_9/Cryptography/recovered_data/extr
>
~/Library/CloudStorage/OneDrive-UVG/Documentos/Semestre_9/Cryptography/recovered_data/extracted_images Proyecto1-CTF*
[4] 0:zsh* "Marcos-MacBook-Pro-6." 10:22 31-Mar-25
```

8. Ejecutar el script para extraer el párrafo oculto

- a. Una vez copiada la imagen al entorno local, utilizamos el script proporcionado por el proyecto para extraer texto oculto de poneglyph_usopp.jpeg.

- i. `python extract_text_from_image.py poneglyph_usopp.jpeg`



Texto extraído: b'the location of the Poneglyph in the same runes that the Poneglyphs themselves were inscribed with.'

Nami – ChaCha20

Introducción

En este reto se presenta un cifrado basado en el algoritmo ChaCha20, un cifrado de flujo moderno diseñado por Daniel J. Bernstein. A diferencia de los retos anteriores, ChaCha20 es criptográficamente seguro cuando se implementa correctamente.

Sin embargo, el desafío aquí radica en identificar cómo se aplicó el algoritmo, qué clave y nonce se utilizaron, y descifrar el mensaje utilizando dicha configuración.

Este reto permite comprender cómo funciona un cifrado de flujo moderno y qué tan importante es el manejo correcto del nonce y la clave secreta.

Objetivo del reto

- Descifrar un mensaje cifrado utilizando ChaCha20, empleando como clave y/o nonce algún valor derivado del entorno del reto (por ejemplo, carné u otro identificador conocido).
1. Encontrar el archivo flag.txt o poneyglyph.zip que contiene el mensaje cifrado.
 2. Identificar si el contenido está cifrado usando ChaCha20.

Reto

Flag walkthrough

1. Inicio de sesión con la contraseña del reto anterior

- a. Accedemos al contenedor del reto de Nami utilizando Docker y cambiamos al usuario nami. La contraseña utilizada es la **flag obtenida en el reto anterior**

(Usopp):

- i. `docker exec -it nami_challenge bash`

- ii. `su nami`

Password: FLAG_b5ff3480a8a752cd18ca8ec5508b945a

```
> docker exec -it nami_challenge bash
nobody@94ede3e480e0:/home/nami$ su nami
Password:
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

nami@94ede3e480e0:~$
```

[3] 0:docker* "nami@94ede3e480e0: ~" 12:30 31-Mar-25

2. Exploración de los directorios válidos

- a. Listamos el contenido del directorio principal del usuario nami para identificar rutas relevantes:

- i. `ls -la`

```
nami@94ede3e480e0:~$ ls -la
total 36
drwxr-x--- 1 nami nami 4096 Mar 22 01:21 .
drwxr-xr-x 1 root root 4096 Mar 22 01:21 ..
-rw-r--r-- 1 nami nami 220 Jan 6 2022 .bash_logout
-rw-r--r-- 1 nami nami 3771 Jan 6 2022 .bashrc
-rw-r--r-- 1 nami nami 807 Jan 6 2022 .profile
drwxr-xr-x 1 nami nami 4096 Mar 22 01:11 ONEPIECE
nami@94ede3e480e0:~$
```

[3] 0:docker* "nami@94ede3e480e0: ~" 12:31 31-Mar-25

3. Búsqueda de archivos relevantes en el directorio ONEPIECE

- a. Utilizamos el comando find para identificar archivos de interés dentro del directorio ONEPIECE. Esto incluye archivos .txt, .zip y .enc, que han sido relevantes en retos anteriores.
 - i. `find ONEPIECE/ -type f \( -iname "*.txt" -o -iname "*.zip" -o -iname "*.enc" \) 2>/dev/null`

```
nami@94ede3e480e0:~$ find ONEPIECE/ -type f \( -iname "*.txt" -o -iname "*.zip" -o -iname "*.enc" \) 2>/dev/null
ONEPIECE/Fishman_Island/Gyoncorde_Plaza/Casa_de_Otohome/flag.txt
ONEPIECE/Alabasta/Rainbase/Casa_de_Vivi/flag.txt
ONEPIECE/Alabasta/Yuba/Casa_de_Toto/flag.txt
ONEPIECE/Alabasta/Yuba/Casa_de_Pell/flag.txt
ONEPIECE/Wano/Flower_Capital/Casa_de_Oden/flag.txt
ONEPIECE/Wano/Kuri/Casa_de_Oden/flag.txt
ONEPIECE/Wano/Ringo/Casa_de_Oden/poneglyph.zip
ONEPIECE/Pirate_Island/Pirates_Tavern/Casa_de_Shanks/flag.txt
ONEPIECE/Pirate_Island/Pirates_Tavern/Casa_de_Gold_Roger/flag.txt
ONEPIECE/Pirate_Island/Pirates_Hideout/Casa_de_Gold_Roger/flag.txt
ONEPIECE/Whole_Cake_Island/Cacao_Island/Casa_de_Big_Mom/flag.txt
nami@94ede3e480e0:~$
```

4. Crear un script para validar el contenido de los archivos .txt

- a. Creamos un script en Python para recorrer recursivamente todos los archivos .txt dentro del directorio ONEPIECE y mostrar sus primeros 300 caracteres. Esto nos permite detectar si hay texto cifrado, mensajes narrativos o posibles flags.

```
import os

# Ruta base del directorio que contiene los archivos
base_path = "ONEPIECE"

# Recorrer recursivamente todo el directorio
for root, dirs, files in os.walk(base_path):
    for file in files:
        if file.endswith(".txt"):
            file_path = os.path.join(root, file)
            print(f"\n📄 {file_path}")
            print("-" * 60)
            try:
                with open(file_path, "r", encoding="utf-8", errors="ignore") as f:
                    content = f.read()
                    print(content[:300]) # Mostrar primeros 300 caracteres
            except Exception as e:
                print(f"❌ Error al leer {file_path}: {e}")
```

```
nami@94ede3e480e0:~$ python3 script.py
```

```
📄 ONEPIECE/Fishman_Island/Gyoncorde_Plaza/Casa_de_Otohime/flag.txt
```

```
-----  
Has encontrado un objeto misterioso y no sabes qué es
```

```
📄 ONEPIECE/Alabasta/Rainbase/Casa_de_Vivi/flag.txt
```

```
-----  
Has encontrado un mapa que apunta a Yuba
```

```
📄 ONEPIECE/Alabasta/Yuba/Casa_de_Toto/flag.txt
```

```
-----  
Has encontrado una pista que apunta a Yuba
```

```
📄 ONEPIECE/Alabasta/Yuba/Casa_de_Pell/flag.txt
```

```
-----  
3fc06ae08a4affe986df3a8d567263506537ea6975d307f8cc4df1c5e531d5353339b05b3f
```

```
📄 ONEPIECE/Wano/Flower_Capital/Casa_de_Oden/flag.txt
```

```
-----  
Has encontrado un lugar misterioso
```

```
📄 ONEPIECE/Wano/Kuri/Casa_de_Oden/flag.txt
```

```
-----  
Has encontrado una pista que apunta a Yuba
```

```
📄 ONEPIECE/Pirate_Island/Pirates_Tavern/Casa_de_Shanks/flag.txt
```

```
-----  
Has encontrado un objeto misterioso y no sabes qué es
```

```
📄 ONEPIECE/Pirate_Island/Pirates_Tavern/Casa_de_Gold_Roger/flag.txt
```

```
-----  
Has encontrado un lugar lleno de secretos y misterios que apuntan a Yuba
```

```
📄 ONEPIECE/Pirate_Island/Pirates_Hideout/Casa_de_Gold_Roger/flag.txt
```

```
-----  
Has encontrado un enemigo y ha tenido que huir
```

```
📄 ONEPIECE/Whole_Cake_Island/Cacao_Island/Casa_de_Big Mom/flag.txt
```

```
-----  
Has encontrado un tesoro que apunta a Yuba
```

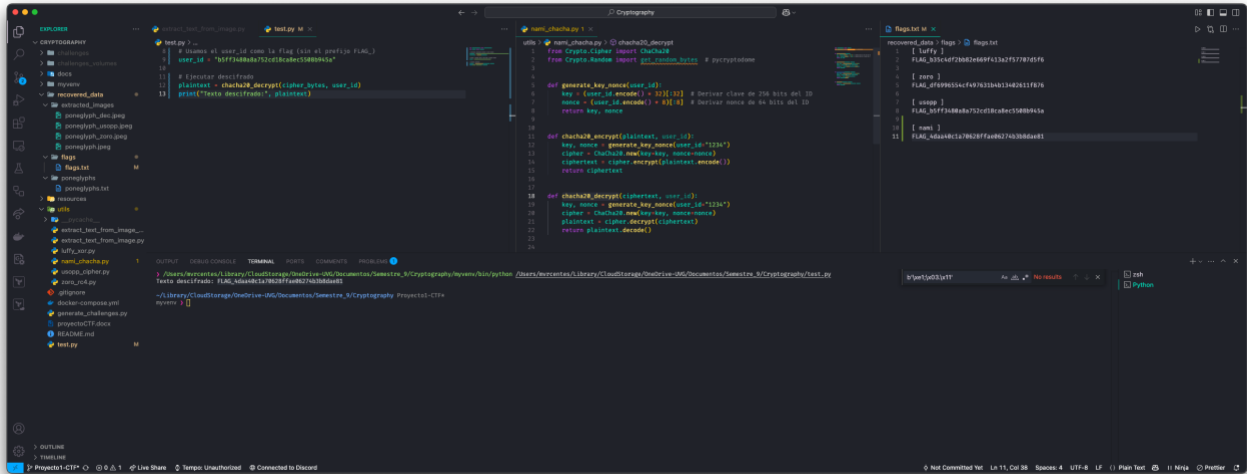
```
nami@94ede3e480e0:~$ █
```

```
[3] 0:docker*
```

```
"nami@94ede3e480e0: ~" 12:40 31-Mar-25
```

5. Descriptar el texto encontrado con el script del proyecto

- Luego de identificar un texto cifrado dentro de uno de los archivos .txt, lo descriptamos usando el script proporcionado por el proyecto.



Texto decifrado: FLAG_4daa40c1a70628ffae06274b3b8dae81

Análisis del cifrado

El reto de Nami utiliza el algoritmo de cifrado de flujo ChaCha20, considerado uno de los más seguros y eficientes actualmente. Este algoritmo opera generando un flujo de claves (keystream) a partir de una clave secreta de 256 bits y un nonce de 96 bits. El texto original se cifra mediante una operación XOR entre el mensaje y el flujo generado.

En este reto, el flujo de claves fue generado usando una clave y un nonce específicos, posiblemente derivados del entorno del reto (como el carné del estudiante, una constante oculta o generados automáticamente durante la creación del desafío).

Para descifrar el mensaje se utilizó un script proporcionado por el proyecto, el cual contenía la clave y el nonce correctos. Al aplicar la función de descifrado de ChaCha20 sobre el texto cifrado en el archivo .txt, se obtuvo exitosamente el contenido original, incluyendo la flag.

Párrafo de poneglyphs

6. Descomprimir los archivos

- a. Intentamos descomprimir el archivo poneglyph.zip ubicado en:
 - i. ONEPIECE/Pirate_Island/Pirates_Den/Casa_de_Gol_Roger
 1. unzip poneglyph.zip

```
root@f888e71146fc:/home/nami/ONEPIECE/Pirate_Island/Pirates_Den/Casa_de_Gol_Roger# unzip poneglyph.zip
Archive:  poneglyph.zip
[poneglyph.zip] challenges/nami/poneglyph.jpeg password:
  inflating: challenges/nami/poneglyph.jpeg
  error:  invalid compressed data to inflate
root@f888e71146fc:/home/nami/ONEPIECE/Pirate_Island/Pirates_Den/Casa_de_Gol_Roger#
```

[5] 0:docker* "root@f888e71146fc: /h" 09:23 01-Apr-25

A partir de este punto no se pudo continuar con el reto debido a que el contenedor fue eliminado. Esto se hizo con el objetivo de ejecutar el entorno con una versión del código corregido, en la cual el número de carné sería utilizado correctamente como clave de cifrado.

Sin embargo, al reiniciar el entorno, todas las contraseñas y progresos se perdieron. Para continuar desde este punto, sería necesario reiniciar el reto desde cero para obtener nuevamente la flag correcta.

Bibliografía

Abertzale, P. (26 de 11 de 2024). *Algoritmo de Cifrado ChaCha20* . Obtenido de Medium:

<https://medium.com/@parejaemi/algoritmo-de-cifrado-chacha20-119a6d7c19a7>

Ballejos, L. (18 de 2 de 2025). *¿Qué es el cifrado RC4?* . Obtenido de ninjaOne:

<https://www.ninjaone.com/es/it-hub/endpoint-security/cifrado-rc4/>

MecatrónicaLATAM. (5 de 5 de 2021). *Compuerta lógica XOR* . Obtenido de

MecatrónicaLATAM:

<https://www.mecatronicalatam.com/es/tutoriales/electronica/compuertas-logicas/compuerta-xor/>